

重大 AI 更新提醒

09-26 | llama.cpp 为 k-quants 引入分块矩阵乘法，大矩阵乘推理提速 3-6 倍

本期导读

本期聚焦 llama.cpp 的一次底层 CPU 推理优化：针对 k-quants 量化格式新增 tiled mul_mat 分块矩阵乘法内核，在大矩阵乘场景下带来 3-6 倍加速，同时保持极低的数值误差。该改动对本地部署量化大模型的开发者具有直接的吞吐收益，但 GEMV（单向量乘矩阵）场景存在约 20% 的性能回退，需要按负载权衡。

已检查来源

GitHub Release

1. llama.cpp b11195：为 k-quants 新增 tiled mul_mat 分块矩阵乘法内核

来源 GitHub Release | 类型 GitHub Release | 主题 检索增强与向量模型 时间 09-26 03:27 | 分数 7

是什么

这是 ggml-org/llama.cpp 的 b11195 版本发布，核心变更是 ggml-cpu 后端为 k-quants 量化格式实现了 tiled mul_mat（分块矩阵乘法）。具体做法是：在每个 mul_mat_one_chunk 中，先把量化权重解包成最大 256x256 的 int8 分块，每种量化类型对应一个解包例程，然后用微内核先计算 16x16 的小块，再把 256x256 的 float 结果写回主内存。该实现配套了 tests/test-tiled-mulmat.cpp 测试与基准，并包含针对 ARM 与 Windows（MSVC）构建的修复，以及 iqpp 代码的首次集成与后续清理。

通俗解释

大模型推理时，最耗时的操作是矩阵乘法，可以理解为 把一大堆数字两两相乘再相加。量化模型为了省内存，把权重压缩成 4bit、5bit 这类低精度格式（k-quants），但计算前必须先解压。过去的做法是边解压边算，缓存不友好、CPU 流水线容易空转。这次改动相当于把一大块权重先切成 256x256 的小方砖，统一解压成 int8 放在离 CPU 更近的地方，再用专门的小内核一次算 16x16 的结果，最后整块写回内存。就像搬砖时先把砖码成整齐的托盘再上叉车，而不是一块一块手递手，因此大块头计算能快 3-6 倍。

主要作用

解决 CPU 上运行 k-quants 量化模型时大矩阵乘性能不足的问题。在本地推理、边缘设备、无 GPU 服务器等场景中，prefill（提示词预填充）和批量推理往往由大矩阵乘主导，该优化直接提升这类负载的吞吐，让量化模型在纯 CPU 环境下更实用。

核心功能

1) 针对 k-quants 的分块解包：将量化权重解包为最大 256x256 的 int8 分块，每种量化类型独立例程；2) 16x16 微内核计算后整块写回 256x256 float 结果，改善缓存局部性；3) 提供 tests/test-tiled-mulmat.cpp 作为可复现的测试与基准；4) 修复 ARM 与 Windows/MSVC 构建问题，并完成 iqpp 代码的首次集成与清理。

对比判断

候选摘要给出的数据：大矩阵乘场景有 3-6 倍速度提升；在 4096x64 乘 64x4096 的规模上达到盈亏平衡点；GEMV（矩阵 向量乘）场景性能约为原来的 80%，即净损失约 20%；数值误差极小，最大误差量级约 1e-04，RMSE 约 1e-05。

适合谁

适合在 CPU 上部署量化大模型的开发者与推理工程师，尤其是使用 llama.cpp 跑 k-quants（如 Q5_K、IQ4_XS 等）格式、关注 prefill 与批量吞吐的团队；也适合研究 ggml 内核优化、量化计算与 SIMD 微内核的研究者，以及需要在 ARM 或 Windows 平台构建的工程团队。

直达链接 <https://github.com/ggml-org/llama.cpp/releases/tag/b11195>

本期仅收录 1 条候选，内容基于 llama.cpp 官方 release 说明；性能数字均来自该 release 摘要，实际收益随模型、量化类型与硬件而异，建议结合自身负载复测。